

От нуля до MVP: задачи React-разработчика

Практический маршрут для проекта `ekt-navigator-react.zip`. Главное в MVP — ИИ-помощник в чате: он отвечает по проверенным данным ЕКТ, помогает выбрать товар и предлагает действие в корзине. Каждый этап заканчивается проверяемым результатом.

Что нужно получить в конце

Клиент начинает с вопроса помощнику: «Есть ли 515291?», «Чем заменить, если нет?», «Как купить?». Ассистент отвечает и показывает карточку товара, подтверждённый аналог или условия покупки. После просьбы добавить товар клиент отдельно подтверждает действие. Поиск и каталог помогают открыть товар вручную, но центральный сценарий проходит через чат.

Уже есть в архиве	Ещё нужно сделать
React + Vite, адаптивная вёрстка и чат на локальных данных	Сделать чат главным экраном и подключить <code>POST /api/chat</code> к Go
Пример карточки ID 515291 и статическое сравнение	Заменить статические данные серверной карточкой и проверенными альтернативами
Локальная демо-корзина	Подтверждение через серверный токен и реальный ответ Go

Фронт никогда не запрашивает ЕКТ API и LLM напрямую. В браузере нет ключей. При отсутствии интеграции с настоящей корзиной показывайте явную пометку «Демо-корзина».

Проверка готовности фронта

- Демонстрация начинается с вопроса в чате; помощник возвращает проверенный товар или честно сообщает, что данных нет.
- Во вкладке Network видны запросы к `/api`; локальные константы больше не используются для цены/остатка в рабочем режиме.
- «Добавь 2» без «Да» не меняет корзину; неизвестный остаток не выглядит как ноль.

Этап 0. Запустить проект в VS Code

- Распаковать ekt-navigator-react.zip. Открыть в VS Code папку ekt-navigator-react, где расположен package.json.
- Проверить node -v и npm -v. Для используемого Vite 7 нужна совместимая Node.js: 20.19+ или 22.12+. Git для локального запуска архива не требуется.

```
npm install
npm run dev
```

Открыть адрес из терминала, обычно `http://localhost:5173`. Сначала задать вопрос в чате, затем проверить карточку, поиск, сравнение и демо-корзину. Перед интеграцией понять, где в `src/App.jsx` находятся `products`, `answer()` и `confirmCart()`. Стили находятся в `src/App.css`.

Результат этапа: текущая демо-страница открывается без ошибки. Если React не запускается, сначала исправьте окружение; Go пока не нужен.

Этап 1. Зафиксировать JSON-контракт с Go

Маршрут вашего Go API	Ответ, который ждёт React
GET /api/search?q=...	{ items: [ProductSummary]}
GET /api/products/{id}	Один нормализованный Product
GET /api/products/{id}/alternatives	{source_product_id, items: [{product, reason, differences}]}
POST /api/chat	{reply, products: [], pending_action: null {product_id, quantity, confirmation_token}}
POST /api/cart/confirm	{status, mode, cart_url?}; 409 если цена/остаток изменились

Внутри Product должны быть `id`, `article`, `name`, `price`, `quantity`, `available_stores`, `characteristics`, `certificate_urls`, `data_issues`. `quantity: null` = неизвестно; `0` = товара нет; положительное число = заявленный остаток. Ошибка: HTTP-код + JSON `{"error": "понятное сообщение"}`.

- Уточнить у бэка: серверный порт; хранение сессии; точные поля `pending_action`; означает ли `mode «live»` настоящую корзину или «demo» локальную.

Результат этапа: оба разработчика согласны с одной таблицей маршрутов и примером ответов. Не начинать подключение корзины до этой договорённости.

Этап 2. Прокси и один модуль API

В vite.config.js направить /api на Go-сервер (пример: localhost:8080). Путь /api сохраняется; на Go должны существовать маршруты с этим префиксом.

```
server: {
  proxy: {
    '/api': {
      target: 'http://localhost:8080',
      changeOrigin: true,
    },
  },
}
```

Создать src/api.js с функцией request(path, options): fetch относительного /api, JSON-разбор, проверка response.ok и ошибка с HTTP-статусом. Если сессия основана на cookie, добавить credentials: "include". Отдельные методы: search, product, alternatives, chat и confirm.

```
export const api = {
  search: (q) => request(`/api/search?q=${encodeURIComponent(q)}`),
  product: (id) => request(`/api/products/${id}`),
  alternatives: (id) => request(`/api/products/${id}/alternatives`),
  chat: (body) => request('/api/chat', {
    method: 'POST', body: JSON.stringify(body),
  }),
  confirm: (token) => request('/api/cart/confirm', {
    method: 'POST',
    body: JSON.stringify({ confirmation_token: token }),
  }),
};
```

Результат этапа: через адрес React запрос /api/chat доходит до Go и возвращает JSON; при остановленном Go видна ошибка. Для ручной проверки можно также открыть /api/search?q=515291.

Этап 3. Данные товара для ответа помощника

- Заменить локальную фильтрацию массива products ответом api.search(query). Пустой запрос может показывать подготовленную серверную выборку; это нужно согласовать с Go.
- При выборе товара вызвать api.product(id). До ответа показать загрузку; при ошибке - сообщение и возможность повторить; после ответа показать поля только из сервера.
- Сохранить различие null и 0. Не вставлять знак валюты до подтверждения валюты партнёром. Сертификат показывать ссылкой лишь при проверенном URL; null = «данные не получены».
- У ID 515291 вывести data_issues: имя/описание 160 А против свойства 250 А. Спорное значение не выводить как установленное.

Результат этапа: карточка 515291 получает свежие поля из Go; поиск остаётся запасным способом открыть её вручную. Помощник будет использовать эту же карточку как источник фактов.

Этап 4. ИИ-помощник и варианты товара

Чат — главный экран MVP

- На отправку сообщения вызвать `api.chat({message, selected_product_id})`. Сохранить `reply` и список структурированных `product`-карточек из ответа.
- Начать демонстрацию с вопроса «Есть ли 515291?»: показать ответ о наличии и карточку. Затем спросить про замену и условия покупки. Чат должен отображать источник неопределённости и не заявлять непроверенные факты.
- Если пришёл `pending_action`, показать отдельный блок: название, ID, количество и подтверждённая цена. Никакого добавления на шаге показа блока.
- На ошибки Go/LLM показывать короткое сообщение; не подменять ответ случайной статической фразой «есть в наличии».

Сравнение внутри чата и карточки

- Если чат вернул проверенные варианты, показать их карточки с `reason` и `differences`. Кнопка «Подобрать замену» может дополнительно вызвать `api.alternatives(id)`.
- Если `items` пустой, сообщить «Проверенный аналог не найден». Наличие общего слова DRX или ID из RECOMM само по себе не означает совместимость.

Результат этапа: свободный вопрос клиента приводит к ответу помощника; карточки и предупреждения совпадают с данными, показанными в каталоге.

Этап 5. Подтверждение и корзина

- По кнопке «Подтвердить» отправить только `confirmation_token` в `api.confirm(token)`. Кнопку блокировать на время запроса, чтобы не сделать повторное добавление.
- При успехе показать `status` и `cart_url` от Go. При `mode: demo` подписать «Демо-корзина»; при `mode: live` можно открыть ссылку на настоящую корзину.
- При HTTP 409 показать изменившийся остаток/цену и попросить новое подтверждение. При ошибке API не увеличивать счётчик корзины локально.

Код/состояние	Что видит клиент
<code>quantity = null</code>	«Наличие неизвестно», добавить нельзя до проверки
<code>quantity = 0</code>	«Нет в наличии» и кнопка подбора аналога
409 Conflict	«Данные изменились», новые условия и запрос нового подтверждения
502/503/timeout	«Не удалось проверить товар сейчас», без изменения корзины

Этап 6. Проверка, сборка, сдача фронта

Сценарий	Что проверить в браузере
Вопрос помощнику	«Есть ли 515291?» возвращает ответ и карточку из Go; нет выдуманной цены или наличия.
Поиск	ID 515291 и артикул 200300285_ ведут к одной карточке.
Качество данных	Ток 160/250 А показан как конфликт, quantity=null не выглядит как 0.
Нулевой остаток	Показываются только проверенные кандидаты, если бэк их нашёл.
Предложение добавить	Сообщение «Добавь 2» не меняет корзину до «Да».
Подтверждение	Один запрос confirm, правильный статус; повторный клик не дублирует.
Сбой Go	Понятная ошибка, старая цена/остаток не заявлены актуальными.
Мобильный экран	Поиск, чат, сравнение и подтверждение доступны без обрезки.

```
npm run build
```

Отдать команде: исходники React, пример настройки прокси, список работающих запросов, скриншоты широкого и мобильного экрана и список того, что пока работает в демо-режиме. README должен объяснять запуск двух терминалов.

Реалистичный порядок на 5 часов

0:00-0:30 - запуск и контракт; 0:30-1:30 - API-клиент и карточка как источник фактов; 1:30-2:30 - работающий чат с ответом Go; 2:30-3:30 - аналоги и подтверждение; 3:30-4:30 - ошибки и мобильный экран; 4:30-5:00 - проверка демо и README. Если времени мало, сначала довести до конца вопрос → ответ → карточка.

Завершено: клиент может задать вопрос ИИ-помощнику и получить ответ по данным Go; неизвестные значения помечены, корзина не меняется без отдельного подтверждения.

Источники: приложенное ТЗ ЕКТ MVP; DATA_CONTRACT.md; официальный Vite Guide - <https://vite.dev/guide/> и server.proxy - <https://vite.dev/config/server-options>.